

Jenny Petrova

February 10 2026

Monte Carlo and quasi-Monte Carlo Methods in Computational Finance Chapter 1 Exercises

(3) Given an LCG with parameters a, c, m , prove that

$$x_{n+k} \equiv a^k x_n + \frac{(a^k - 1)}{a - 1} c \pmod{m}; \quad (a \geq 2, k \geq 0)$$

which shows that the $(n + k)$ th term can be computed directly from the n th term.

Proof. We will do a proof by induction. First, we must establish the base case. Take $k = 0$ and solve:

$$\begin{aligned} x_n &= a^0 x_n + \frac{(a^0 - 1)}{a - 1} c \pmod{m} \\ &= (1)x_n + \frac{(1 - 1)}{a - 1} c \pmod{m} \\ &= x_n \pmod{m}. \end{aligned}$$

Now suppose the formula,

$$x_{n+k} \equiv a^k x_n + \frac{(a^k - 1)}{a - 1} c \pmod{m}; \quad (a \geq 2, k \geq 0),$$

holds for some fixed k . We must show the above holds for $k + 1$. Consider the following:

$$\begin{aligned} x_{n+k+1} &= ax_{n+k} + c \pmod{m} \\ &= a \left(a^k x_n + \frac{(a^k - 1)}{a - 1} c \right) + c \pmod{m} \\ &= a^{k+1} x_n + \left(\frac{a(a^k - 1)}{a - 1} + 1 \right) c \pmod{m} \\ &= a^{k+1} x_n + \left(\frac{a^{k+1} - a}{a - 1} + \frac{a - 1}{a - 1} \right) c \pmod{m} \\ &= a^{k+1} x_n + \frac{(a^{k+1} - 1)}{a - 1} c \pmod{m}. \end{aligned}$$

Therefore, by induction the formula holds for all $k \geq 0$. □

- (6) Write a code that would implement RANDU. For debugging purposes, print x_{100} when the seed is $x_0 = 1$.

```
1 # Output for randutest(100):
2 1673794513
```

- a. Using RANDU generate $u_1, \dots, u_{20,002}$, where $u_n = x_n/M$. For all triplets in your sequence, (u_i, u_{i+1}, u_{i+2}) , in which $0.50 \leq u_{i+1} \leq 0.51$, plot u_i versus u_{i+2} . Comment on the pattern of your scatterplot.

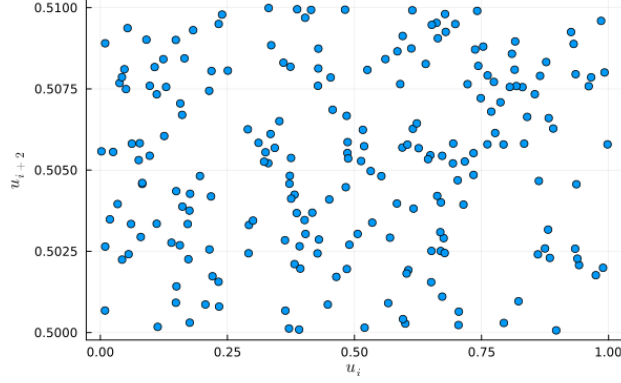


Figure 1: 2D scatterplot of u_i vs u_{i+2} for $\{u_i\}_1^{20002}$ RANDU sequence

From visual observation, the above 2D scatterplot *appears* random. However, we see the points exhibit noticeable clustering and large gaps across the plot, suggesting a potential issue of strong dependence between successive values of the RANDU generator.

- b. Generate a sequence of length 1002. Use a program that plots points in 3 dimensions and rotates the axes to rotate the points until you can see the 15 planes. (S-Plus function *spin* may be helpful.)

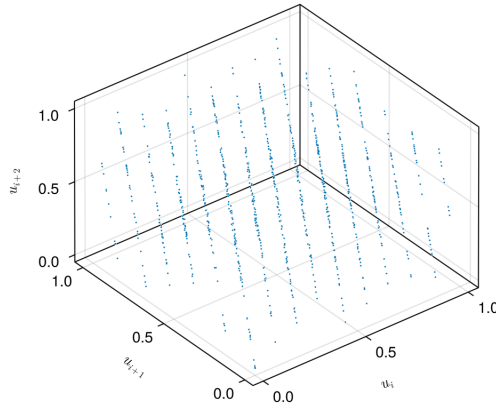


Figure 2: 3D scatterplot of u_i vs u_{i+1} vs u_{i+2} for $\{u_i\}_1^{20002}$ RANDU sequence

Examining the above 3D scatterplot, we notice something particularly suspicious: the points lie on 15 two-dimensional parallel planes. Ideally, we would expect such pseudorandomly-generated points to uniformly fill the unit cube. However, our results imply the RANDU generator does *not* produce truly random points. This supports our previous observations, where we noted the issue of strong dependence between successive outputs, hence suggesting RANDU is poor random number generator.

(8) Prove the second inequality of Lemma 1.1 for $s = 2$.

Lemma 1.1

For any finite set $P = \{q_1, \dots, q_N\}$, we have

$$D_N^*(P) \leq D_N(P) \leq 2^s D_N^*(P).$$

Proof. Consider the definition of discrepancy and star-discrepancy for $s = 2$:

$$D_N(q_n) = \sup_{[a,b]} \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{\Pi_{i=1}^2[a_i, b_i]}(q_n) - \lambda_2([a, b]) \right|, \quad D_N^*(q_n) = \sup_{[0,a]} \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{\Pi_{i=1}^2[0, a_i]}(q_n) - \lambda_2([0, a]) \right|$$

We begin with the expansion of the first term in the discrepancy. Let $q_n = (x_n, y_n) \in [0, 1]^2$ and observe that

$$\begin{aligned} \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{\Pi_{i=1}^2[a_i, b_i]}(q_n) &= \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[a_1, b_1] \times [a_2, b_2]}(x_n, y_n) \\ &= \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[a_1, b_1]}(x_n) \mathbf{1}_{[a_2, b_2]}(y_n) \\ &= \frac{1}{N} \sum_{n=1}^N \left((\mathbf{1}_{[0, b_1]}(x_n) - \mathbf{1}_{[0, a_1]}(x_n)) (\mathbf{1}_{[0, b_2]}(y_n) - \mathbf{1}_{[0, a_2]}(y_n)) \right) \\ &= \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, b_1] \times [0, b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, b_2]}(x_n, y_n) \\ &\quad - \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_2] \times [0, b_1]}(x_n, y_n) + \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, a_2]}(x_n, y_n). \end{aligned}$$

Now, we expand the second term in the discrepancy, such that

$$\lambda_2([a, b]) = \prod_{i=1}^2 [a_i, b_i] = (b_1 - a_1)(b_2 - a_2) = b_1 b_2 - a_1 b_2 - b_1 a_2 + a_1 a_2.$$

We combine the above terms and apply the triangle inequality to show

$$\begin{aligned} &\left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{\Pi_{i=1}^2[a_i, b_i]}(q_n) - \lambda_2([a, b]) \right| \\ &= \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, b_1] \times [0, b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_2] \times [0, b_1]}(x_n, y_n) \right. \\ &\quad \left. + \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, a_2]}(x_n, y_n) - b_1 b_2 + a_1 b_2 + b_1 a_2 - a_1 a_2 \right| \\ &\leq \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, b_1] \times [0, b_2]}(x_n, y_n) - b_1 b_2 \right| + \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, b_2]}(x_n, y_n) - a_1 b_2 \right| \\ &\quad + \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_2] \times [0, b_1]}(x_n, y_n) - b_1 a_2 \right| + \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{[0, a_1] \times [0, a_2]}(x_n, y_n) - a_1 a_2 \right|, \end{aligned}$$

Therefore, taking the supremum of the above result over all $0 \leq a < b \leq 1$ yields

$$D_N(q_n) \leq 4D_N^*(q_n) = 2^2 D_N^*(q_n).$$

□

(10) The following is a test function commonly used in numerical integration

$$f(x^{(1)}, \dots, x^{(s)}) = \prod_{i=1}^s \frac{\pi}{2} \sin(\pi x^{(i)}).$$

The integral of f over $(0,1)^s$ is equal to 1. Estimate this integral using MC and RQMC methods by computing

$$I_k = \frac{1}{N} \sum_{n=1}^N f(q_n^{(1)}, \dots, q_n^{(s)})$$

where $(q_n^{(1)}, \dots, q_n^{(s)})^T$ is the n th term of an s -dimensional randomly shifted Halton sequence in the case of RQMC, and pseudorandom vectors generated using the Mersenne Twister in the case of MC. The subscript k of I_k refers to the k th estimate for the integral using the k th randomly shifted Halton sequence (RQMC) or Mersenne Twister (MC).

- Use Mersenne Twister to obtain $I_1, \dots, I_m$ where $m = 40, N = 1000, s = 10$.
- Use randomly shifted Halton sequences to obtain $I_1, \dots, I_m$ where $m = 40, N = 1000, s = 10$.
- Repeat parts (a) and (b) with $N = 10,000$.
- Use box-whisker plots to display the estimates you obtained in (a) through (c) together with the exact value. Which method seems to be better in approximating the integral?

We fix seed = 1234 for the following results.

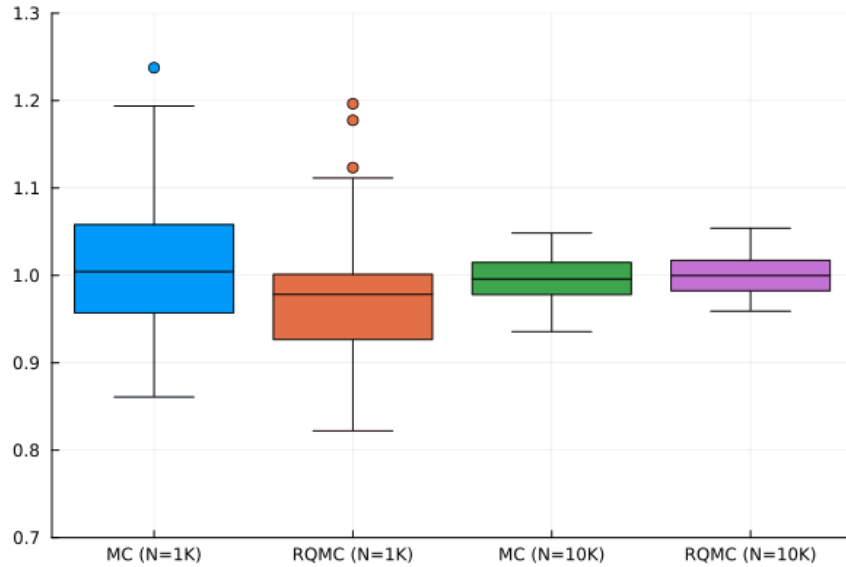


Figure 3: Box-and-whisker plots for MC vs RQMC for $m = 40$ estimates

Observing the above box-whisker plots for $m = 40$ integral estimates, $\{I_1, I_2, \dots, I_{39}, I_{40}\}$, there is a significant improvement in the estimation, for both methods, when increasing the number of generated sequences (N). For $N = 10000$ sequences of $s = 10$ pseudorandomly-generated variables, the RQMC method produces a tighter spread than the MC method, suggesting improvements in the estimation and convergence to the true value of the integral. Notice that for $N = 1000$ and $N = 10000$, the MC method yields a median value *closer* to the true value of the integral. Despite this, the latter observation is misleading, since we want to consider estimation accuracy through the general trend in the distribution of the estimates, rather than from one single median estimate in our distribution.

Appendix

```
1      using Random, Primes
2  using Pkg
3  using StatsPlots
4  using Plots
5  using LaTeXStrings
6  using GLMakie
7  using Plots
8  using GLMakie
9  GLMakie.activate!()
10
11  # -----
12  # Chapter 1 Problem 6
13  # -----
14
15  # ----- 6 (a) -----
16
17  # test function for RANDU with seed x0=1
18  function randutest(n,x=1)
19      M = 2^(31)
20      for i in 1:n
21          x = mod(65539 * x, M) # RANDU generator
22      end
23      return x
24  end
25  x = randutest(100)
26  print(x)
27
28  # complete RANDU generator
29  function randu(n,x=1)
30      seq = zeros(n)
31      u_lower = []
32      u_upper = []
33      M = 2^(31)
34      for i in 1:n
35          seq[i] = x # store generated points
36          x = mod(65539 * x, M) # RANDU generator
37      end
38      u = seq / M
39      for i in 1:n
40          # if  $0.5 \leq u_{i+1} \leq 0.51$ , then store  $u_{i+1}$  and  $u_{i+2}$  as coordinate pairs
41          if u[i] <= 0.51 && u[i] >= 0.50
42              push!(u_lower, u[i-1])
43              push!(u_upper, u[i])
44          end
45      end
46      return u, u_lower, u_upper
47  end
48
49  u1, u_lower, u_upper = randu(20002)
50
51  # create 2D scatterplot of  $u_{i+1}$  vs  $u_{i+2}$ 
52  Plots.scatter(u_lower, u_upper,
53      xlabel = L"$u_{i+1}$",
54      ylabel = L"$u_{i+2}$",
55      title = "RANDU",
56      legend = false
57  )
58
```

```

59 # ----- 6 (b) -----
60
61 u2, _, _ = randu(1002)
62
63 # store consecutive triplets
64 x = u2[1:1000]
65 y = u2[2:1001]
66 z = u2[3:1002]
67
68 # create 3D (interactive) scatterplot of triplets (u_{i} vs u_{i+1} vs u_{i+2})
69 fig = Figure()
70 ax = Axis3(fig[1, 1], xlabel = L"$u_i$", ylabel = L"$u_{i+1}$", zlabel = L"$u_{i+2}$")
71 GLMakie.scatter!(ax, x, y, z; markersize = 2)
72 display(fig)
73
74 # -----
75 # Chapter 1 Problem 10
76 # -----
77
78 m = 40
79 N1 = 1000
80 N2 = 10000
81 s = 10
82
83 # create generator for MC method
84 rng = MersenneTwister(1234)
85
86 # create generator for RQMC method
87 lprimes=primes(2,200)
88
89 # generate nth element of the van der Corput sequence in base b
90 function vdc(n,b)
91     digs=digits(n,base=b)
92     sum=0.0
93     for i in 1:length(digs)
94         sum = sum + digs[i]/b^i
95     end
96     return(sum)
97 end
98
99 # get the nth element of the s-dimensional Halton sequence
100 function halton(n,s)
101     [vdc(n,i) for i in lprimes[1:s]]
102 end
103
104 # randomly shift the s-dimensional Halton sequence
105 function shifthalton(n,s)
106     u = rand(rng, s)
107     map(k->mod(k,1),halton(n,s).+u)
108 end
109
110 # estimate the integral for Nxs matrix with N pseudorandomly generated s-dimensional sequences
111 function fintegral(N, s, method="rqmc")
112     I = 0.0
113     for i in 1:N
114         if method=="mc"
115             q = rand(rng, s)
116         else
117             q = shifthalton(i, s)
118         end

```

```

119     term = 1.0
120     for j in 1:(length(q))
121         term *= (pi/2) * sin(pi * q[j])
122     end
123     I += term
124 end
125 return (I/N)
126 end
127
128 # print integral estimates for MC and RQMC, for N=1000 and N=10,000
129 println("Integral Estimates for N=$N1")
130 I_mc1 = [fintegral(N1, s, "mc") for _ in 1:m]
131 println("Mersenne Twister: $I_mc1")
132 I_rqmc1 = [fintegral(N1, s) for _ in 1:m]
133 println("Randomly shifted Halton: $I_rqmc1")
134 println("Integral Estimates for N=$N2")
135 I_mc2 = [fintegral(N2, s, "mc") for _ in 1:m]
136 println("Mersenne Twister: $I_mc2")
137 I_rqmc2 = [fintegral(N2, s) for _ in 1:m]
138 println("Randomly shifted Halton: $I_rqmc2")
139
140 # create box-and-whisker plots for each set of integral estimates
141 data=[I_mc1, I_rqmc1, I_mc2, I_rqmc2]
142 Plots.boxplot(data, legend=false)
143 Plots.xticks!(1:4, ["MC (N=1K)", "RQMC (N=1K)", "MC (N=10K)", "RQMC (N=10K)"])
144 Plots.ylims!(0.7,1.3)

```

```

1 # OUTPUTS
2 x_100 for seed x_0 = 1: 1673794513
3
4 Integral Estimates for N=1000
5 Mersenne Twister: [1.0058202890799817, 0.9769243020628176, 1.0570478031767925, ... ,
6   ↳ 0.9527564458898824, 0.8422999154302471, 0.982975562154701]
7 Randomly shifted Halton: [1.0193026427777465, 0.8838394069112271, 0.9818815387351807, ... ,
8   ↳ 0.9485128515993083, 0.863356384226958, 0.9551964029770618]
9
10 Integral Estimates for N=10000
11 Mersenne Twister: [0.9782856665256597, 0.9900870158560825, 0.9866250138358466, ... ,
12   ↳ 1.0119412909144712, 0.990465861619219, 0.9478905442609182]
13 Randomly shifted Halton: [1.0168483207214152, 1.0217322205223915, 1.0140601481747953, ... ,
14   ↳ 1.0397986894956894, 1.0125711653268537, 1.006734289853648]

```

Alternative Solutions

(3) Given an LCG with parameters a, c, m , prove that

$$x_{n+k} \equiv a^k x_n + \frac{(a^k - 1)}{a - 1} c \pmod{m}; \quad (a \geq 2, k \geq 0)$$

which shows that the $(n + k)$ th term can be computed directly from the n th term.

Proof. Lehmer's LCG is defined as

$$x_{n+1} \equiv ax_n + c \pmod{m}.$$

Now, suppose we want to compute x_{n+2} from x_n . Consider the following:

$$\begin{aligned} x_{n+2} &\equiv ax_{n+1} + c \pmod{m} \\ &= a(ax_n + c) + c \pmod{m} \\ &= a^2 x_n + (a + 1)c \pmod{m}. \end{aligned}$$

Let's go one step further! Compute x_{n+3} from x_n :

$$\begin{aligned} x_{n+3} &\equiv ax_{n+2} + c \pmod{m} \\ &= a(a^2 x_n + (a + 1)c) + c \pmod{m} \\ &= a^3 x_n + (a^2 + a + 1)c \pmod{m}. \end{aligned}$$

If we continue the above recursions up to x_{n+k} , we obtain

$$\begin{aligned} x_{n+k} &\equiv ax_{n+k-1} + c \pmod{m} \\ &= a^k x_n + \left(\sum_{i=0}^{k-1} a^i \right) c \pmod{m} \\ &= a^k x_n + \left(\frac{1 - a^{(k-1)+1}}{1 - a} \right) c \pmod{m}. \\ &= a^k x_n + \left(\frac{1 - a^k}{1 - a} \right) c \pmod{m}. \\ &= a^k x_n + \left(\frac{a^k - 1}{a - 1} \right) c \pmod{m}, \end{aligned}$$

where the summation is a finite geometric series. □